

Jeux video IA

Chapitre 01 :

Fondamentaux des jeux vidéo

Focus du chapitre:

Comprendre comment un jeu vidéo fonctionne comme un système interactif, et comment l'IA y permet de modéliser des agents capables de percevoir, décider et agir.

Pr. Youness BOUTYOUR

Plan du chapitre

- I Jeu vidéo comme système interactif
- II Boucle de jeu : exécution et dynamique
- III Notions clés : état, action, règles, objectifs, récompenses
- IV Place de l'IA dans un jeu vidéo
- V Types d'agents dans les jeux

Objectifs pédagogiques

- Comprendre le jeu vidéo comme un système interactif dynamique
- Identifier les notions fondamentales : état, action, règle, objectif, récompense
- Situer la place de l'intelligence artificielle dans la boucle de jeu
- Distinguer les principaux types d'agents : scriptés, réactifs, adaptatifs, évolutifs
- Relier ces notions à des cas d'usage concrets : navigation, prise de décision, optimisation

PLAN DE LA SECTION

I. Jeu vidéo comme système interactif

- ① Définition d'un jeu vidéo
- ② Composantes d'un système interactif
- ③ Cycle interaction joueur-jeu
- ④ Exemples simples de jeux

Qu'est-ce qu'un jeu vidéo ?

□ Un jeu vidéo est un système numérique interactif dans lequel :

- un joueur (ou plusieurs) effectue des actions ;
- ces actions modifient un monde simulé ;
- le système applique des règles (contraintes + logique) ;
- le jeu fournit un feedback (visuel/sonore/score) ;
- l'expérience est orientée vers un objectif (défi, progression, exploration).

Un jeu vidéo = système interactif (joueur/agents) + environnement simulé + règles + objectifs + feedback.

Qu'est-ce qu'un jeu vidéo ?

□ Un jeu vidéo est un système numérique interactif dans lequel :

- un joueur (ou plusieurs) effectue des actions ;
- ces actions modifient un monde simulé ;
- le système applique des règles (contraintes + logique) ;
- le jeu fournit un feedback (visuel/sonore/score) ;
- l'expérience est orientée vers un objectif (défi, progression, exploration).

Un jeu vidéo = système interactif (joueur/agents) + environnement simulé + règles + objectifs + feedback.

Exemple du jeu Serpent:

état=position, action=direction, règle=collision, objectif=score.

Qu'est-ce qui transforme un programme en jeu vidéo ?



□ Un jeu vidéo se distingue par :

- **interaction continue** (boucle joueur-jeu) ;
- **dynamique temporelle** (temps réel ou tours) ;
- **règles ludiques** (victoire/défaite, progression, score) ;
- **incertitude / événementiel** (obstacles, adversaires, hasard parfois) ;
- **expérience utilisateur** (fun, challenge, immersion).

❑ Un jeu vidéo se distingue par :

- **interaction continue** (boucle joueur-jeu) ;
- **dynamique temporelle** (temps réel ou tours) ;
- **règles ludiques** (victoire/défaite, progression, score) ;
- **incertitude / événementiel** (obstacles, adversaires, hasard parfois) ;
- **expérience utilisateur** (fun, challenge, immersion).

❑ Les 5 éléments incontournables

- **Joueur / Agents** (qui agit ?)
- **Environnement** (où cela se passe ?)
- **Actions** (que peut-on faire ?)
- **Règles** (qu'est-ce qui est autorisé / interdit / produit ?)
- **Feedback** (comment le joueur perçoit ce qui arrive ?)

Qu'est-ce qu'un système interactif ?

❑ Un système interactif est un système qui :

- reçoit des entrées (input),
- réalise un traitement (processing),
- produit des sorties (output),
- et ajuste son comportement en fonction des actions de l'utilisateur.

❑ Dans un jeu :

- entrées = contrôles / événements
- traitement = moteur + règles + IA
- sorties = rendu + audio + score + sensations

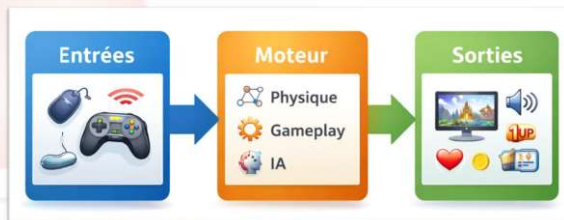


Architecture "Entrées-Traitement-Sorties"

❑ Modèle fonctionnel du jeu

- **Entrées** : clavier, souris, manette, réseau
- **Traitement** :
 - moteur (physique, collisions, animation)
 - règles du gameplay
 - IA (agents, navigation, décision)
- **Sorties** : images, sons, UI, score, feedback

Modèle fonctionnel



Composants internes (vue moteur de jeu)

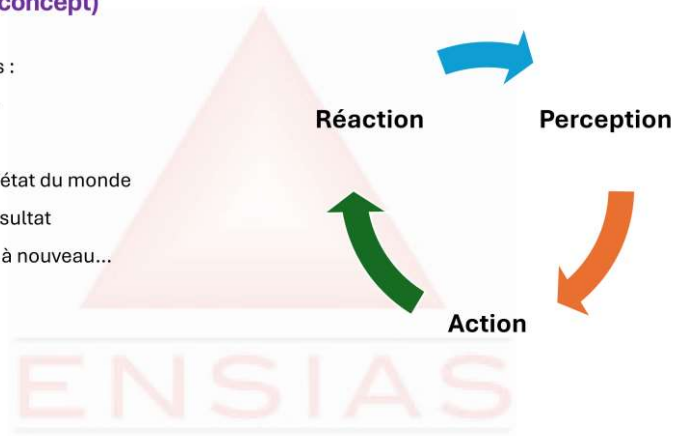
❑ Ce que contient généralement un jeu :

Gestion du monde	carte, objets, entités
Physique	mouvements, interactions
Règles	scoring, victoire, contraintes
Animation / Rendu	visuel, audio, UI
IA	comportement, pathfinding, décision
Gestion des événements	triggers, scripts, quêtes

Cycle d'interaction (concept)

❑ Le jeu évolue par cycles :

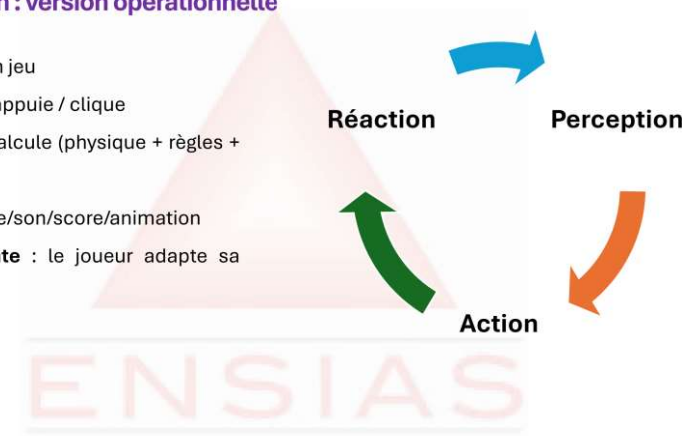
1. le joueur **observe**
2. le joueur **agit**
3. le jeu **met à jour** l'état du monde
4. le jeu **affiche** le résultat
5. le joueur **observe** à nouveau...



Boucle d'interaction : version opérationnelle

❑ Cycle concret dans un jeu

- **Input** : le joueur appuie / clique
- **Update** : le jeu calcule (physique + règles + IA)
- **Feedback** : image/son/score/animation
- **Décision suivante** : le joueur adapte sa stratégie

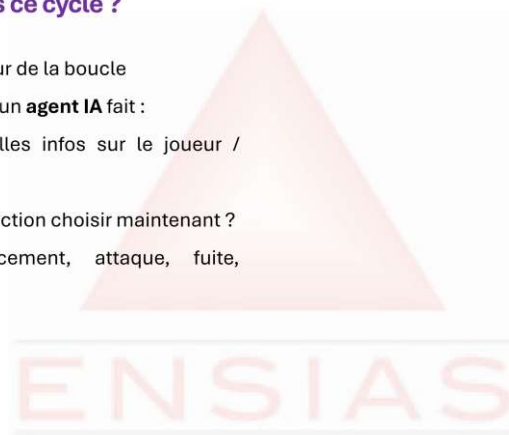


Où se place l'IA dans ce cycle ?

❑ IA = décision à l'intérieur de la boucle

❑ Pendant la mise à jour, un **agent IA** fait :

- **perception** : quelles infos sur le joueur / environnement ?
- **décision** : quelle action choisir maintenant ?
- **action** : déplacement, attaque, fuite, interaction...



Où se place l'IA dans ce cycle ?

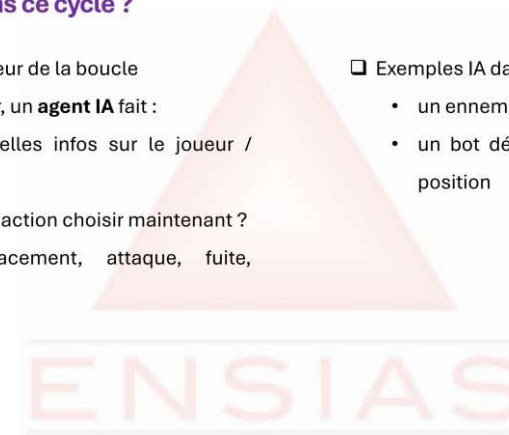
❑ IA = décision à l'intérieur de la boucle

❑ Pendant la mise à jour, un **agent IA** fait :

- **perception** : quelles infos sur le joueur / environnement ?
- **décision** : quelle action choisir maintenant ?
- **action** : déplacement, attaque, fuite, interaction...

❑ Exemples IA dans le cycle :

- un ennemi "voit" le joueur → poursuit
- un bot décide de se couvrir → change de position



Exemple 1 : Serpent (décomposition rapide)

❑ Serpent comme système interactif

- Entrées :
- Traitement :
 -
 -
 -
- Sorties :



Exemple 1 : Serpent (décomposition rapide)

❑ Serpent comme système interactif

- Entrées : changement de direction
- Traitement :
 - mise à jour de la position
 - collision mur/corps
 - génération nourriture
- Sorties : grille affichée, score, fin de partie, ...



Exemple 2 : Pac-Man (décomposition rapide)

❑ Pac-Man comme système interactif

- Entrées :
- Traitement :
 -
 -
 -
- Sorties :



Exemple 2 : Pac-Man (décomposition rapide)

❑ Pac-Man comme système interactif

- Entrées : déplacement du joueur
- Traitement :
 - déplacements des fantômes
 - collisions, bonus, score
 - règles
- Sorties : animations, score, vies, sons,



PLAN DE LA SECTION

II. Boucle de jeu

- ① Pourquoi une boucle de jeu ?
- ② Déroulement d'une frame
- ③ Gestion du temps et performance
- ④ Intégration de l'IA dans la frame

II. Boucle de jeu

1. Pourquoi une boucle de jeu ?

Un jeu est un système dynamique exécuté par itérations

□ À chaque **itération**, il :

- lit les entrées,
- met à jour l'état,
- résout la simulation,
- affiche le résultat

Objectif:

Garantir une **évolution temporelle cohérente** du système sous contraintes de calcul.

□ La boucle de jeu impose un schéma d'exécution itératif qui :

- cohérence (ordre d'exécution stable),
- réactivité (contrôle fluide),
- stabilité (éviter comportements "aléatoires")

II. Boucle de jeu

1. Pourquoi une boucle de jeu ?

Discretisation du temps et simulation

□ Un jeu exécute une simulation numérique. En pratique :

- Le temps est **échantillonné** en pas discrets :

$$t_0, t_1, \dots, t_k$$

- L'état évolue selon une dynamique :

$$s_{t+1} = f(s_t, a_t, \Delta t)$$

où :

- s_t : état courant,
- a_t : actions (joueur + agents),
- Δt : durée de l'itération.

II. Boucle de jeu

1. Pourquoi une boucle de jeu ?

Propriétés attendues

□ Ce qu'on attend d'une boucle de jeu :

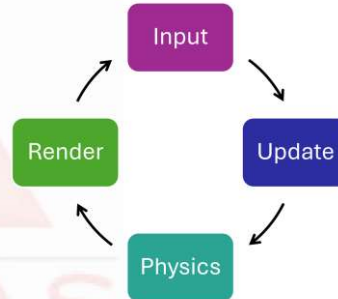
- Réactivité : faible latence entre action et feedback
- Stabilité : simulation physique cohérente malgré variations de FPS
- Déterminisme partiel : reproductibilité utile pour debug/test
- Budget de calcul : chaque frame doit tenir dans un temps limité
- Modularité : intégrer IA, physique, animation, audio, UI

Architecture d'une frame (vue macro)

❑ Pipeline d'une frame typique

1. Input : capture des entrées (joueur/réseau)
2. Update : mise à jour logique (mouvements, scripts, IA)
3. Physics/Collisions/Events : résolution (collisions, triggers, scoring)
4. Render : affichage + audio + UI

Remarque : selon le moteur, le découpage exact varie, mais le principe reste.



Pseudocode standard

```

while (jeu_en_cours) {
    lire_entrees();
    mettre_a_jour_etat();
    appliquer_regles();
    gerer_collisions_evenements();
    afficher();
}
  
```

Input : acquisition et filtrage

Étape 1 — Input

❑ Rôle :

- collecter les entrées (clavier, souris, manette, réseau),
- transformer en commandes interprétables (ex. axes analogiques),
- gérer événements (press/release), buffers et priorités.

❑ Points importants :

- l'input est temporel (séquences),
- l'input peut être bruité (manette, réseau),
- certaines actions doivent être échantillonnées (ex. click unique).

Update : logique et état

Étape 2 — Update

❑ Update applique :

- règles gameplay (score, vie, inventaire),
- scripts/événements,
- animations logiques,
- IA (perception → décision),
- gestion d'objectifs, quêtes, transitions.

Formulation :

$$s_{t+1}^{(logic)} = g(s_t, u_t)$$

où u_t regroupe commandes et événements.

Physique & collisions

Étape 3 — Physique / collisions

❑ Objectifs :

- intégrer le mouvement,
- détecter collisions,
- résoudre contacts (réactions),
- garantir cohérence spatiale.

❑ Pourquoi c'est séparé ?

- La physique peut exiger un pas de temps fixe (stabilité),
- Les collisions sont coûteuses → optimisations.

Rendu : sortie perceptive

Étape 4 — Rendu

❑ Le Rendu produit :

- image (caméra, shaders),
- audio,
- UI,
- effets visuels.

• Important

- « Rendu » n'est pas la simulation, c'est la représentation.

Δt, FPS et budget temps

Temps de frame et FPS (Frame Per Second)

Si le jeu tourne à *FPS*, alors :

$$\Delta t \approx \frac{1}{FPS}$$

❑ Exemples :

- 60 FPS → $\Delta t \approx 16.67$ ms
- 30 FPS → $\Delta t \approx 33.33$ ms

Conséquence : toute la logique (physique + IA + règles) doit tenir dans ce budget.

Fixed timestep vs variable timestep

Deux stratégies principales

(A) Pas de temps variable:

- Δt change selon la charge
- simple, mais peut déstabiliser la physique

(B) Pas de temps fixe (fixed timestep):

- simulation à pas constant (ex. 20 ms)
- rendu peut interpoler entre états
- plus stable, mais demande une architecture plus rigoureuse

Latence : délai action → feedback

❑ La latence totale inclut :

- acquisition input,
- traitement logique,
- simulation,
- rendu,
- affichage.

❑ En temps réel :

- latence élevée = contrôle "lourd",
- impact direct sur jouabilité et perception de qualité.

❑ Objectif pratique : minimiser la latence perçue.

Où s'insère l'IA ?

❑ Dans un jeu, l'IA doit produire des décisions sous contraintes :

- **Perception** : collecte features (distance, visibilité, état joueur)
- **Décision** : sélection action (tirer, fuir, patrouiller)
- **Action** : déclenchement (mouvement, animation, tir)

❑ L'IA est un composant de la mise à jour logique du jeu.

❑ Elle doit produire des actions cohérentes et réactives sous contraintes temps réel.

❑ Problème central :

- comment exécuter **Perception → Décision → Action (PDA)** sans dépasser le budget temps?

Modèle PDA (Perception–Decision–Action)

Cycle interne d'un agent IA (PDA)

❑ Pour chaque agent i à la frame t :

- Perception : $o_t^i = \phi(s_t)$ (extraction d'observation depuis l'état)
- Décision : $a_t^i = \pi(o_t^i)$ (politique / contrôleur)
- Action : exécution + effets via le moteur (animation/physique/règles)

Interprétation:

- ϕ = capteurs (vision, distance, bruit,)
- π = logique IA (BT, planning, RL, heuristiques)

Où l'IA se place dans le pipeline (vue frame)

Pipeline recommandé (conceptuel) :

1. Input joueur
2. Mise à jour monde (pré-IA) : timers, événements, états internes
3. Perception IA : collecte features / capteurs
4. Décision IA : sélection d'intentions et actions
5. Application actions : commandes moteur (move, shoot, interact)
6. Physique + collisions
7. Post-update : scoring, triggers, synchronisation
8. Render

Faut-il décider à chaque frame ?

Fréquence de décision (decision rate)

❑ Décider à chaque frame n'est pas toujours nécessaire :

- Perception : souvent fréquente ($\approx$ chaque frame ou quasi)
- Décision : peut être plus lente (toutes les k frames)
- Planification : encore plus lente (à l'événement, ou à intervalle)

❑ On peut définir :

$$a_t^i = \begin{cases} \pi(o_t^i) & \text{si } t \bmod k = 0 \\ a_{t-1}^i & \text{sinon (hold / smoothing)} \end{cases}$$



π peut être :

- règles (if/then),
- machine à états (FSM),
- behavior tree (BT),
- planification (ex. GOAP),
- politique apprise (RL, imitation learning),
- ou une combinaison hiérarchique.

Intuition : réduire les recalculs coûteux sans réduire la crédibilité.

Budget temps réel et coût par agent

Budget IA : contrainte quantitative

- À 60 FPS, budget frame $\approx 16,7$ ms.
- Si on réserve B_{IA} ms à l'IA, alors pour N agents :

$$\text{coût moyen par agent} \leq \frac{B_{IA}}{N}$$

- IA doit être **hiérarchisée** et **approximée** (sinon explosion du coût).

PLAN DE LA SECTION

III. Notions clés

- ① État et observation
- ② Actions et espace d'action
- ③ Règles, transitions et dynamique
- ④ Objectifs et récompenses

III. Notions clés

1. État et observation

Définition — État s_t

❑ L'état s_t est une représentation (potentiellement complète) de la situation du jeu à l'instant t , contenant les variables nécessaires pour :

- simuler l'évolution du monde,
- décider des actions,
- évaluer les résultats.

❑ Exemples de variables d'état :

- positions, vitesses, collisions en cours
- HP, munitions, inventaire
- score, temps restant, objectifs actifs

État global vs état local

- ❑ **État global** : description complète du monde (utile pour le moteur).
- ❑ **État local** : sous-ensemble pertinent pour une entité (utile pour l'agent).

Un agent n'a pas forcément accès à toutes les informations (réalisme, gameplay).

Définition : observation o_t^i

- ❑ Pour un agent i , l'observation est l'information accessible à cet agent à l'instant t :

$$o_t^i = \phi(s_t)$$

où ϕ est une fonction de perception (capteurs, visibilité, bruit, mémoire).

Exemples :

- line-of-sight (**LOS**) : joueur visible ou non
- distance estimée à la cible
- dernier point connu du joueur

Pourquoi distinguer état et observation ?

- ❑ Raisons principales:

- **Réalisme** : un agent ne "voit" pas à travers les murs.
- **Gameplay** : éviter une IA "injuste".
- **Complexité** : réduire la dimension d'entrée de la décision.
- **Modélisation** : prépare les cadres partiellement observables.

Exemple simple :

- état global : position exacte du joueur
- observation : "bruit entendu à droite", "joueur non visible"

Définition — Action a_t

- ❑ Une action est un choix appliqué par un joueur ou un agent au temps t qui influence l'évolution du jeu.

$$a_t \in \mathcal{A}$$

où $\mathcal{A}$ est l'ensemble des actions possibles.

Espace d'action $\mathcal{A}$

- ❑ **Actions discrètes** (ensemble fini)

$$\mathcal{A} = \{\text{gauche, droite, sauter, tirer}\}$$

- ❑ **Actions continues** (valeurs réelles)

- $\mathcal{A} \subset \mathbb{R}^d$
- ex. angle de visée, intensité accélération, vitesse

Bon design de l'espace d'action

- ❑ Un bon espace d'action doit être :
- **expressif** : permet les comportements voulus
 - **contrôlable** : évite explosion combinatoire
 - **aligné** avec les animations/physique
 - **stable** : évite oscillations d'actions

ENSIAS

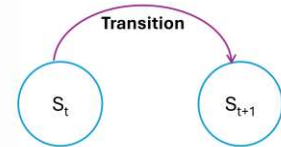
Définition : transition

- ❑ Dynamique du jeu
- ❑ La transition décrit comment l'état évolue :

$$s_{t+1} = f(s_t, a_t, \Delta t)$$

où :

- f intègre physique + collisions + règles gameplay,
- Δt est le pas de temps (frame time ou fixed step).



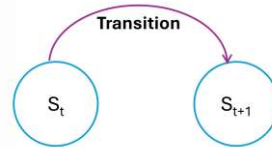
ENSIAS

Règles : ce que "fait" f

- ❑ Les règles définissent :
- ce qui est autorisé / interdit,
 - les effets des actions,
 - les interactions (collisions, dégâts),
 - les conditions d'arrêt (mort, fin niveau, victoire).

Exemple :

- "si collision avec mur → bloquer mouvement"
- "si projectile touche → dégâts"



ENSIAS

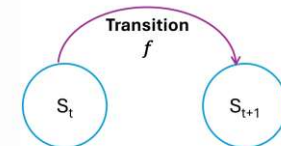
Transitions déterministes vs stochastiques

- ❑ Deux types de dynamique
- **Déterministe** : même action dans même état → même résultat.
 - **Stochastique** : présence d'aléa (bruit, critique, spawn).
- ❑ On peut formaliser :

$$P(s_{t+1} | s_t, a_t)$$

Pourquoi c'est utile ?

- modéliser hasard contrôlé
- éviter IA trop "parfaite"
- améliorer rejouabilité



ENSIAS

Définition — objectif

❑ Un objectif est un critère de réussite du point de vue :

- du joueur,
- du game design,
- ou d'un agent autonome.

Exemples:

- gagner une manche,
- survivre,
- maximiser un score,
- minimiser le temps,
- capturer une zone.

Définition — Récompense r_t

❑ La récompense est un signal numérique (souvent) associé au résultat d'une action :

$$r_t = R(s_t, a_t, s_{t+1})$$

❑ Elle sert à :

- mesurer la qualité d'un comportement,
- guider l'apprentissage (si on utilise RL),
- comparer des stratégies.

Définition — Récompense r_t

❑ La récompense est un signal numérique (souvent) associé au résultat d'une action :

$$r_t = R(s_t, a_t, s_{t+1})$$

❑ Elle sert à :

- mesurer la qualité d'un comportement,
- guider l'apprentissage (si on utilise RL),
- comparer des stratégies.

Concevoir une récompense efficace

- ❑ Bénéfice : accélérer l'apprentissage / guider le comportement.
- ❑ Risque : "récompense trichable" → l'agent optimise un proxy non désiré.

Exemples :

- si récompense = "survivre", l'agent peut ne rien faire (camping)
- si récompense = "être proche de la cible", il peut tourner en rond.

Principe : récompenser ce qu'on veut réellement obtenir, pas un indicateur ambigu.

PLAN DE LA SECTION

IV. Place de l'IA dans un jeu vidéo

- ① Rôles et objectifs de l'IA dans les jeux
- ② Positionnement de l'IA dans l'architecture du jeu

Définition opérationnelle : "IA de jeu"

❑ Dans le contexte jeu vidéo, "IA" désigne l'ensemble des méthodes permettant à des entités non-joueurs de :

- percevoir l'état/observation,
- décider d'actions,
- agir de manière cohérente,

Avec des objectifs centrés sur l'expérience :

- crédibilité,
- défi,
- immersion,
- variété,
- jouabilité.



IA comme instrument de contrôle du gameplay

❑ L'IA est souvent conçue pour :

- guider la tension dramatique,
- contrôler la difficulté,
- orienter le joueur sans le forcer,
- maintenir le rythme (combat/exploration/respiration).



Les 4 grandes fonctions de l'IA (vue design)

- ❑ Challenge : créer un adversaire intéressant
- ❑ Crédibilité : comportements plausibles (illusion d'intelligence)
- ❑ Rejouabilité : variabilité des situations et stratégies
- ❑ Assistance : coéquipiers, tutorat, aides contextuelles

IA : "performance" vs "believability"

❑ Deux axes majeurs :

- Performance (efficacité) : gagner, optimiser, atteindre un objectif
- Believability (crédibilité) : être cohérent, lisible, "humain"

❑ En jeu vidéo, on cherche souvent un compromis :

- un ennemi trop parfait → frustration ("injuste")
- un ennemi trop faible → ennui



Où s'exécute l'IA ? (vue système)

❑ Dans le pipeline d'une frame, l'IA apparaît surtout dans :

- Update (logique) : perception → décision → commandes
- Systèmes de navigation : recherche de chemin + évitement
- Systèmes d'animation : exécution visuelle d'actions décidées

Architecture logicielle typique (composants IA)

❑ Composants IA fréquents dans un moteur de jeu :

- **Perception System** : LOS, raycasts, zones, bruit
- **Decision System** : FSM / Behavior Tree / Utility AI
- **Navigation System** : graph/navmesh + steering
- **Blackboard / Memory** : dernière position vue, menace, objectifs
- **Animation/Actuation** : locomotion, actions, blends



IV. Place de l'IA dans un jeu vidéo

2. Positionnement de l'IA dans l'architecture du jeu

IA individuelle vs IA de "système"

□ Deux niveaux:

- IA d'entité (micro) : un ennemi, un allié
- IA de directeur (macro) : gestion du pacing, spawns, difficulté, ressources (souvent appelée Director AI ou Game Manager)

PLAN DE LA SECTION

V. Types d'agents

- | | |
|--|--------------------------------------|
| ① Notion d'agent dans un jeu vidéo | ⑤ Agents adaptatifs |
| ② Agents scriptés | ⑥ Agents évolutifs / apprenants |
| ③ Agents réactifs | ⑦ Comparaison et choix de conception |
| ④ Agents délibératifs / basés décision | |

V. Types d'agents

1. Notion d'agent et niveaux d'autonomie

Définition :

□ Un agent est une entité capable de :

- Percevoir : obtenir une observation o_t ,
- Décider : choisir une action a_t ,
- Agir : exécuter dans l'environnement.

□ Schéma formel :

$$o_t = \phi(s_t), \quad a_t = \pi(o_t)$$

- ϕ : perception (capteurs, features)
- π : politique/contrôleur (règles, arbre, plan, modèle appris)

V. Types d'agents

1. Notion d'agent et niveaux d'autonomie

Degré d'autonomie : un continuum

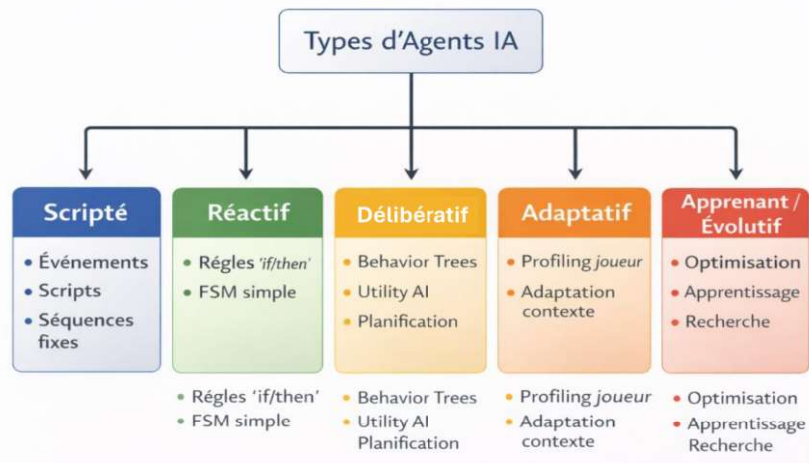
□ Les agents se placent sur un continuum :

Scripté → Réactif → Délibératif → Adaptatif → Apprenant

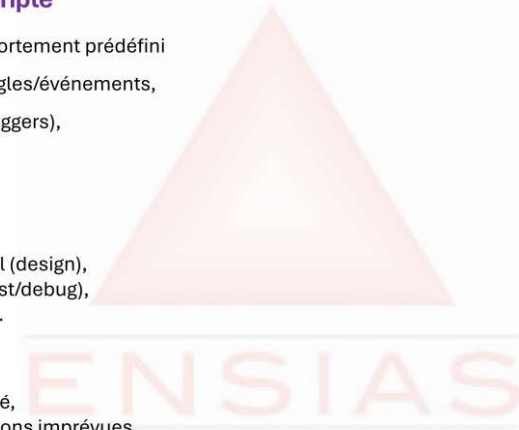
- plus on va à droite :
 - plus l'agent est flexible,
 - mais moins il est "contrôlable",
 - et plus il est coûteux à concevoir/calculer.

□ Critères d'évaluation d'un agent

- Pour juger un agent (au-delà de "il gagne") :
 - **Crédibilité** (believability) : cohérence, lisibilité
 - **Efficacité** : atteint l'objectif
 - **Robustesse** : gère les cas limites (stuck, occlusion)
 - **Contrôle designer** : réglage simple, comportements prévisibles
 - **Coût** : CPU/GPU, mémoire, complexité d'implémentation

**Définition : agent scripté**

- ☐ Agent scripté = comportement prédéfini
 - séquences de règles/événements,
 - déclencheurs (triggers),
 - scripts narratifs,
 - patterns fixes.
- ☐ **Avantages**
 - contrôle maximal (design),
 - reproductible (test/debug),
 - très peu coûteux.
- ☐ **Limites**
 - prévisible,
 - faible adaptabilité,
 - fragile aux situations imprévues.

**Définition : agent scripté**

- ☐ Agent scripté = comportement prédéfini
 - séquences de règles/événements,
 - déclencheurs (triggers),
 - scripts narratifs,
 - patterns fixes.
- ☐ **Avantages**
 - contrôle maximal (design),
 - reproductible (test/debug),
 - très peu coûteux.
- ☐ **Limites**
 - prévisible,
 - faible adaptabilité,
 - fragile aux situations imprévues.
- ☐ **Usage typique**
 - narration, tutoriel, puzzles, événements scénarisés.
- ☐ **Exemple :**

Garde en patrouille:

 - suit une liste de points $A \rightarrow B \rightarrow C \rightarrow A$
 - déclenche une animation à chaque point
 - si joueur détecté : alerte (script)
- ☐ **Remarque :** même avec un déclencheur, la logique reste essentiellement scriptée.

Définition : agent réactif

- ☐ Agent réactif = stimulus → réponse
 - décisions locales,
 - règles conditionnelles (if/then),
 - faible ou pas de planification.
- ☐ **Forme typique :**
 - si "joueur visible" → poursuivre
 - si "hp faible" → fuir
- ☐ **Implémentations fréquentes :**
 - règles if/then,
 - systèmes de règles,
 - FSM simples (machines à états finis).
- ☐ **Atout majeur :** temps réel, rapide, robuste en coût.



Définition : agent réactif

- ❑ Agent réactif = stimulus → réponse
 - décisions locales,
 - règles conditionnelles (if/then),
 - faible ou pas de planification.
- ❑ Forme typique :
 - si "joueur visible" → poursuivre
 - si "hp faible" → fuir
- ❑ Implémentations fréquentes :
 - règles if/then,
 - systèmes de règles,
 - FSM simples (machines à états finis).
- ❑ Atout majeur : temps réel, rapide, robuste en coût.

- ❑ Limites (réactif):
 - myopie : pas d'anticipation,
 - oscillation : change d'état à chaque frame,
 - comportement artificiel : "trop mécanique".
- ❑ Solutions pratiques :
 - hystérésis,
 - cooldown,
 - actions duratives,
 - décisions toutes les k frames.

Définition : agent délibératif

- ❑ Délibératif = décision structurée + objectifs
 - prend des décisions en considérant des alternatives,
 - peut utiliser :
 - hiérarchie,
 - planification,
 - utilité (scores),
 - arbres de comportement.
- ❑ **Objectif** : comportements flexibles tout en restant lisibles.

Définition : agent délibératif

- ❑ Délibératif = décision structurée + objectifs
 - prend des décisions en considérant des alternatives,
 - peut utiliser :
 - hiérarchie,
 - planification,
 - utilité (scores),
 - arbres de comportement.
- ❑ **Objectif** : comportements flexibles tout en restant lisibles.

- ❑ Un agent délibératif choisit des actions en évaluant :
 - le contexte (observation o_t),
 - des objectifs explicites,
 - et/ou une structure de décision (hiérarchique ou planifiée).

Formellement :

$$a_t = \pi(o_t, m_t, g_t)$$

où :

- o_t : observation,
- m_t : mémoire/blackboard,
- g_t : objectif (goal) ou intention.

Architecture typique (hiérarchie)

- ❑ Décision structurée : "intention → exécution"
 - **Niveau haut (délibération)** : choisir l'intention
 - ex. attaquer, se couvrir, fuir, chercher, défendre une zone
 - **Niveau bas (réaction/exécution)** : réaliser l'intention.
 - ex. navigation locale, visée, évitement, animation
- ❑ **Bénéfices**
 - stabilité (moins d'oscillations),
 - lisibilité (comportements compréhensibles),
 - modularité (remplacer un sous-comportement).
- ❑ **Exemples de techniques:**
 - BT
 - Utility
 - Planification
 -

Behavior Trees (BT : Arbre de comportement) : principe

❑ Un **BT** est un arbre composé de :

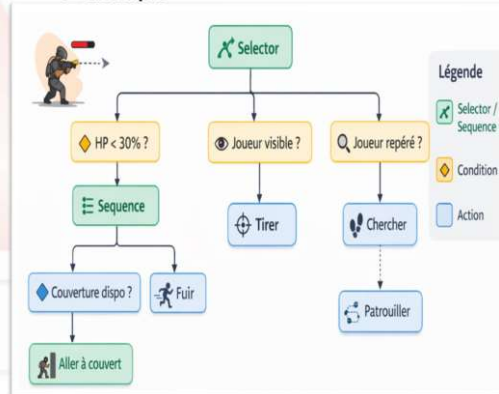
- nœuds de contrôle : Sequence, Selector, Parallel
- conditions : tests (LOS, distance, HP)
- actions : tirer, se déplacer, se couvrir, patrouiller

❑ **Idée** : exécuter l'arbre périodiquement ("tick") pour produire une action.

❑ **Bénéfices**

- très utilisé en production (lisible, débogable, modulaire).

❑ **Exemple**



Utility AI : principe

❑ Choisir l'action par score

❑ Pour chaque action a , calculer une utilité :

$$U(a | o_t) \in \mathbb{R}$$

❑ Choisir:

$$a^* = \arg \max_a U(a | o_t)$$

❑ **Exemples de features** : distance à la cible, HP, munitions, couverture, menace.

❑ **Atout principal** : décisions fluides et adaptatives (pas "tout ou rien").

Utility AI : principe

❑ Choisir l'action par score

❑ Pour chaque action a , calculer une utilité :

$$U(a | o_t) \in \mathbb{R}$$

❑ Choisir:

$$a^* = \arg \max_a U(a | o_t)$$

❑ **Exemples de features** : distance à la cible, HP, munitions, couverture, menace.

❑ **Atout principal** : décisions fluides et adaptatives (pas "tout ou rien").

❑ **Exemple simplifié : 3 actions, 3 scores**

❑ **Actions** :

- a_1 : tirer
- a_2 : se couvrir
- a_3 : fuir

❑ **Utilités (ex. simplifié)** :

- $U(\text{tirer}) = w_1 \cdot \text{visibilité} + w_2 \cdot (1 - \text{distance})$
- $U(\text{couvert}) = w_3 \cdot (1 - \text{HP}) + w_4 \cdot \text{couverture}$
- $U(\text{fuir}) = w_5 \cdot (1 - \text{HP}) + w_6 \cdot \text{menace}$

❑ **Observation** :

tuning des poids w_i = réglage designer.

Planification (GOAP / HTN)

❑ La planification consiste à construire automatiquement une séquence d'actions pour atteindre un objectif (goal) à partir d'un état initial.

$$\text{Plan} = [a_1, a_2, \dots, a_k] \text{ tel que } s_0 \xrightarrow{a_1, \dots, a_k} s_g$$

GOAP (Goal-Oriented Action Planning)

❑ **Principe** : actions définies par **préconditions** et **effets** (et parfois un **coût**).

❑ Action a :

- Préconditions : $Pre(a)$
- Effets : $Eff(a)$
- Coût : $c(a)$

❑ Le planificateur cherche un plan **minimisant le coût total** : $\min \sum_{j=1}^k c(a_j)$

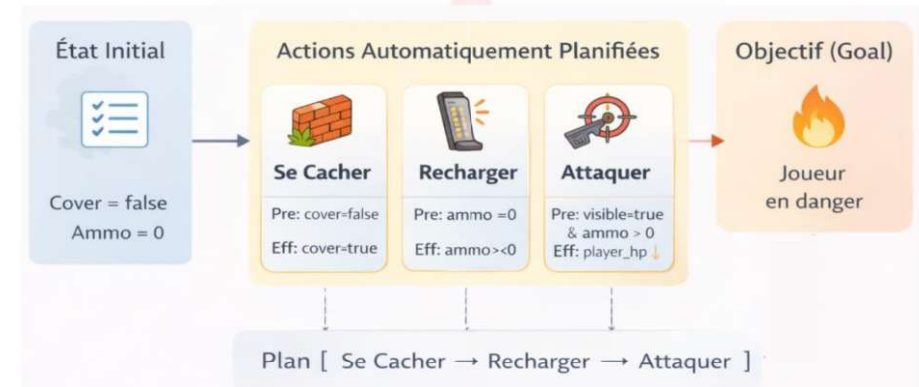
Exemple (ennemi IA)

Goal : mettre le joueur en danger

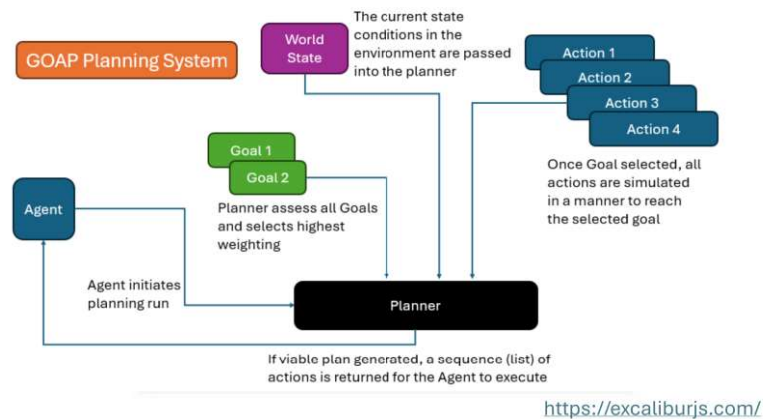
Actions :

- se mettre à couvert (Pre: cover=false, Eff: protected=true)
- recharger (Pre: ammo=0, Eff: ammo>0)
- attaquer (Pre: visible=true & ammo>0, Eff: player_hp↓)

GOAP (Goal-Oriented Action Planning)



GOAP (Goal-Oriented Action Planning)



HTN (Hierarchical Task Networks)

❑ **Principe** : on décompose un objectif en **sous-tâches** (hiérarchie).

❑ **Tâche** : *Sécuriser la zone*

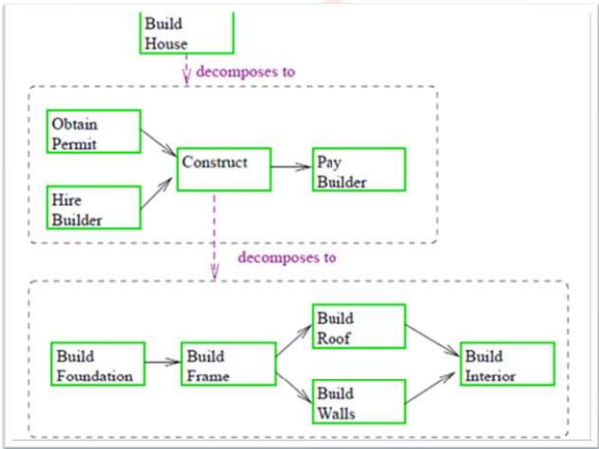
- Reconnaître → Se positionner → Neutraliser menace → Patrouiller

❑ **Atouts**:

- très contrôlable par les designers,
- mieux adapté à des comportements "scénarisés mais flexibles".

HTN (Hierarchical Task Networks)

❑ Exemple:



Source

Définition : évolutif / apprenant

❑ Deux grandes approches :

- ❑ **Évolutif (optimisation offline)**
 - algorithmes évolutionnaires / recherche
 - sélection de paramètres/comportements
- ❑ **Apprenant (learning)**
 - imitation learning,
 - apprentissage par renforcement (RL),
 - modèles qui approximent $\pi(a|o)$

Définition : évolutif / apprenant

❑ Deux grandes approches :

- ❑ **Évolutif (optimisation offline)**
 - algorithmes évolutionnaires / recherche
 - sélection de paramètres/comportements
- ❑ **Apprenant (learning)**
 - imitation learning,
 - apprentissage par renforcement (RL),
 - modèles qui approximent $\pi(a|o)$

Pourquoi apprendre ? Pourquoi c'est difficile ?

- ❑ **Atouts**
 - émergence de stratégies inattendues,
 - automatisation du tuning,
 - adaptation à des environnements complexes.
- ❑ **Difficultés**
 - coût d'entraînement,
 - contrôle designer plus faible,
 - comportements non anticipés,
 - généralisation et robustesse.
- ❑ En jeu commercial, on combine souvent :
 - apprentissage offline + contraintes designers,
 - ou modèles appris encapsulés dans une logique BT/FSM.

Comparaison synthétique

Type d'agent	Contrôle (design)	Flexibilité	Coût (runtime)	Risque d'imprévu	Cas d'usage typique
Scripté	Très élevé	Faible	Très faible	Faible	cinématiques, tutoriels, patterns fixes
Réactif	Élevé	Moyen	Faible	Moyen	ennemis simples, foule, réflexes
Délibératif (BT/Utility/Planning)	Moyen	Élevé	Moyen	Moyen	ennemis/boss riches, PNJ crédibles
Adaptatif	Moyen	Élevé	Moyen	Élevé	personnalisation, difficulté dynamique
Évolutif / Apprenant	Faible→Moyen	Très élevé	variable	Élevé	bots, tuning offline, émergence

Références pour approfondir

Architecture des jeux, boucle de jeu, IA et design ludique

Livres

Millington, I. & Funge, J. — AI for Games
3e éd., Routledge, 2019

Gregory, J. — Game Engine Architecture
3e éd., CRC Press / A K Peters, 2018

Schell, J. — The Art of Game Design: A Book of Lenses
3e éd., CRC Press, 2019

Salen Tekinbaş, K. & Zimmerman, E. — Rules of Play
Game Design Fundamentals, MIT Press, 2003

Ressources web

Game Programming Patterns — « Game Loop »
gameprogrammingpatterns.com

Game AI Pro
gameai.pro

Unity Documentation — AI Navigation
docs.unity3d.com

Unreal Engine Documentation — Behavior Trees
dev.epicgames.com

Références pour approfondir

Architecture des jeux, boucle de jeu, IA et design ludique

Livres

